

GCD Acceleration Controller

GCD :- Greatest Common Divisor.

GCD is the largest positive integer that divides two or more integers without leaving a remainder.

Ex

$$\text{GCD}(12, 18) = 6.$$

Divisor of 12 :- 1, 2, 3, 4, 6, 12

Divisor of 18 :- 1, 2, 3, 6, 9, 18

Common Divisor = 1, 2, 3, 6

The greatest Common divisor (GCD) = 6

Modulo operator

$$20 \% 8$$

$$20 = (8 \times 2) + 4$$

$$\therefore 20 \% 8 = 4$$

Euclidean Algorithm

The Euclidean Algorithm is an efficient method for finding the Greatest Common Divisor (GCD) of two integers using repeated remainder operations.

Formula

$$\text{GCD}(A, B) = \text{GCD}(B, A \% B)$$

Algorithm

Calculate $R = A \% B$

Assign $A = B$

Assign $B = R$

* Repeat the process until B becomes 0

* When B becomes 0, A contains the GCD.

Hardware Implementation

In hardware, the Euclidean algorithm can be implemented using registers, remainder/modulo logic, and a control FSM.

Working of Algorithm

$$17 \div (17 \cdot 5)$$

Based on Algorithm

$$A = 17$$

$$B = 5$$

$$R = A \% B$$

$$R = 2$$

Next value updation

$$A = B$$

$$A = 5$$

$$B = R$$

$$B = 2$$

Next iteration

$$A = 5$$

$$B = 2$$

Again

$$5 \% 2 = 1$$

update

$$A = 2$$

$$B = 1$$

Update

$$A = 1$$

$$B = 0$$

Again

$$2 \% 0 = 0$$

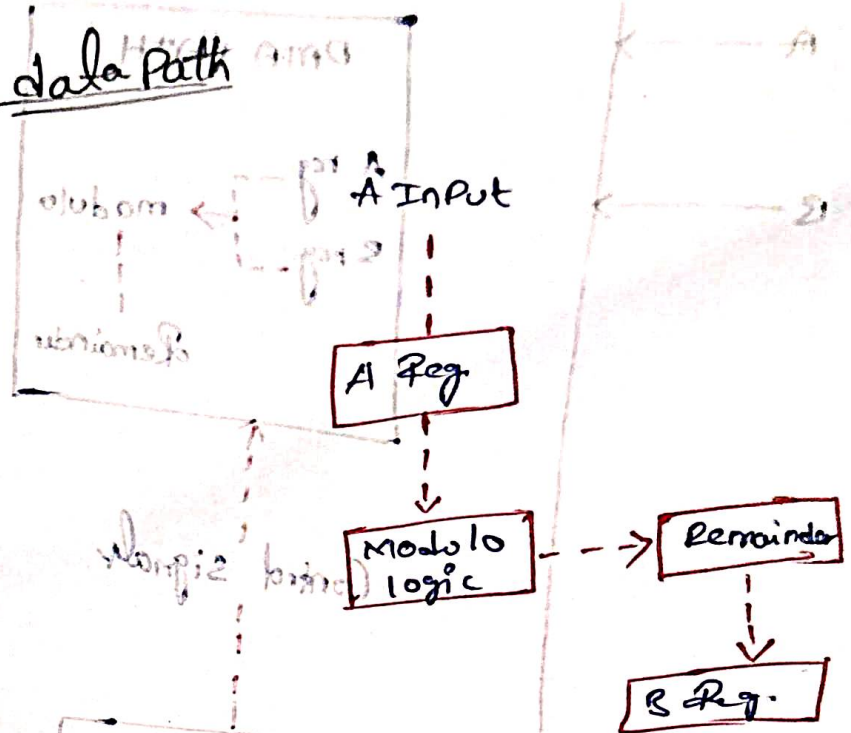
$B == 0$,
So
Stop.
GCD = 1

① Data Path

A data path is the hardware that stores and processes actual data

In GCD Project data Path

- A register
- B register
- Remainder logic
- Result register.



② Control Unit

The control unit controls the sequence of operations using an FSM and generates control signals for the datapath

Ex:-

Load
Calculate
Update
Check
Done

Imagine like a
States.

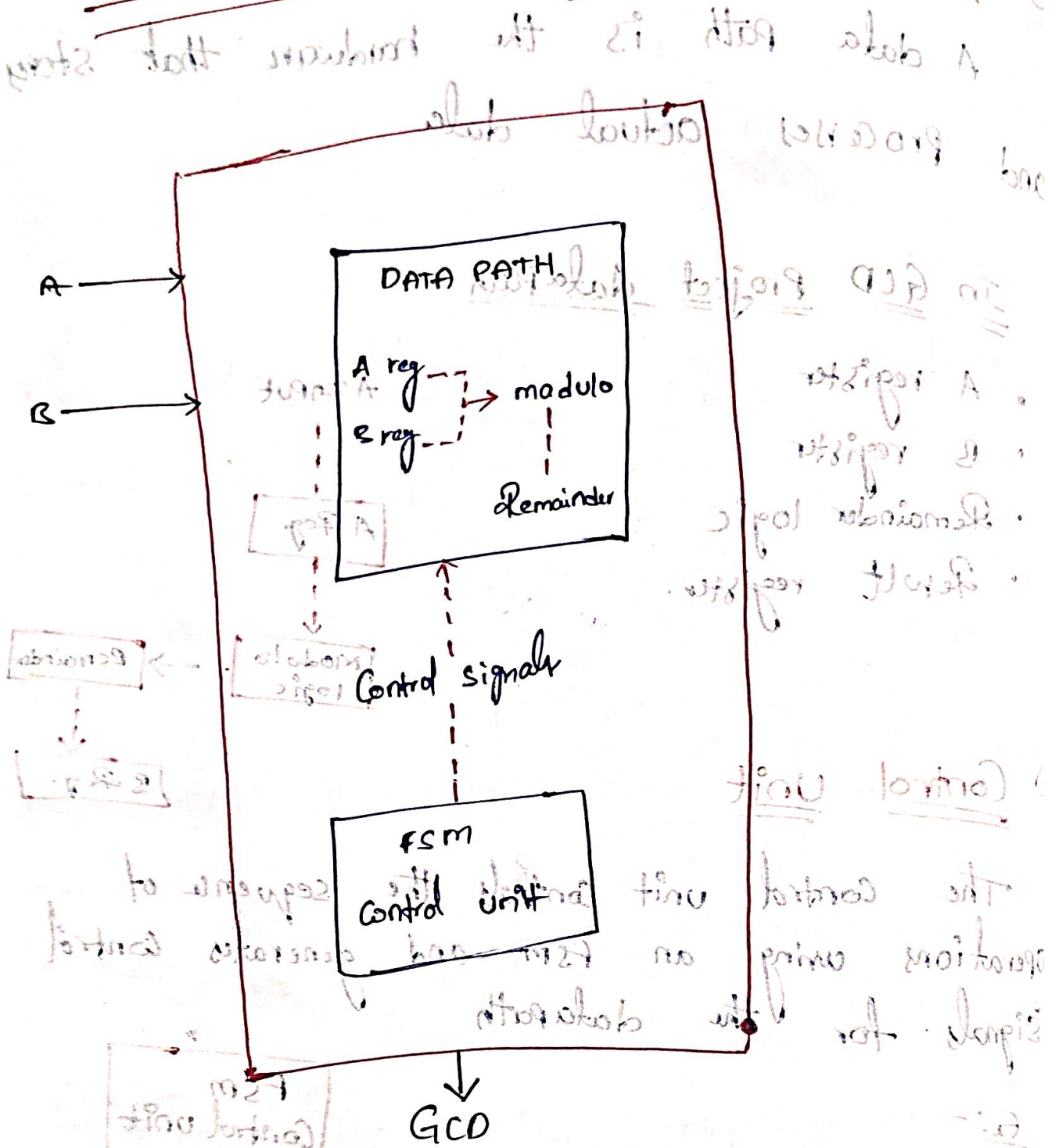
FSM
Control unit

Control signal

Data Path

* Control units are usually implemented by FSM's

3. DATAPATH + CONTROL UNIT

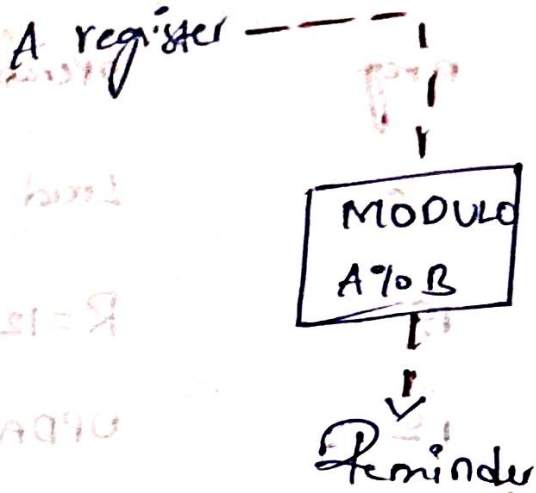


Data path = data processing
 Control unit = decision/control

Handwritten notes at the bottom of the page, including "GCD" and other illegible text.

Reminder logic

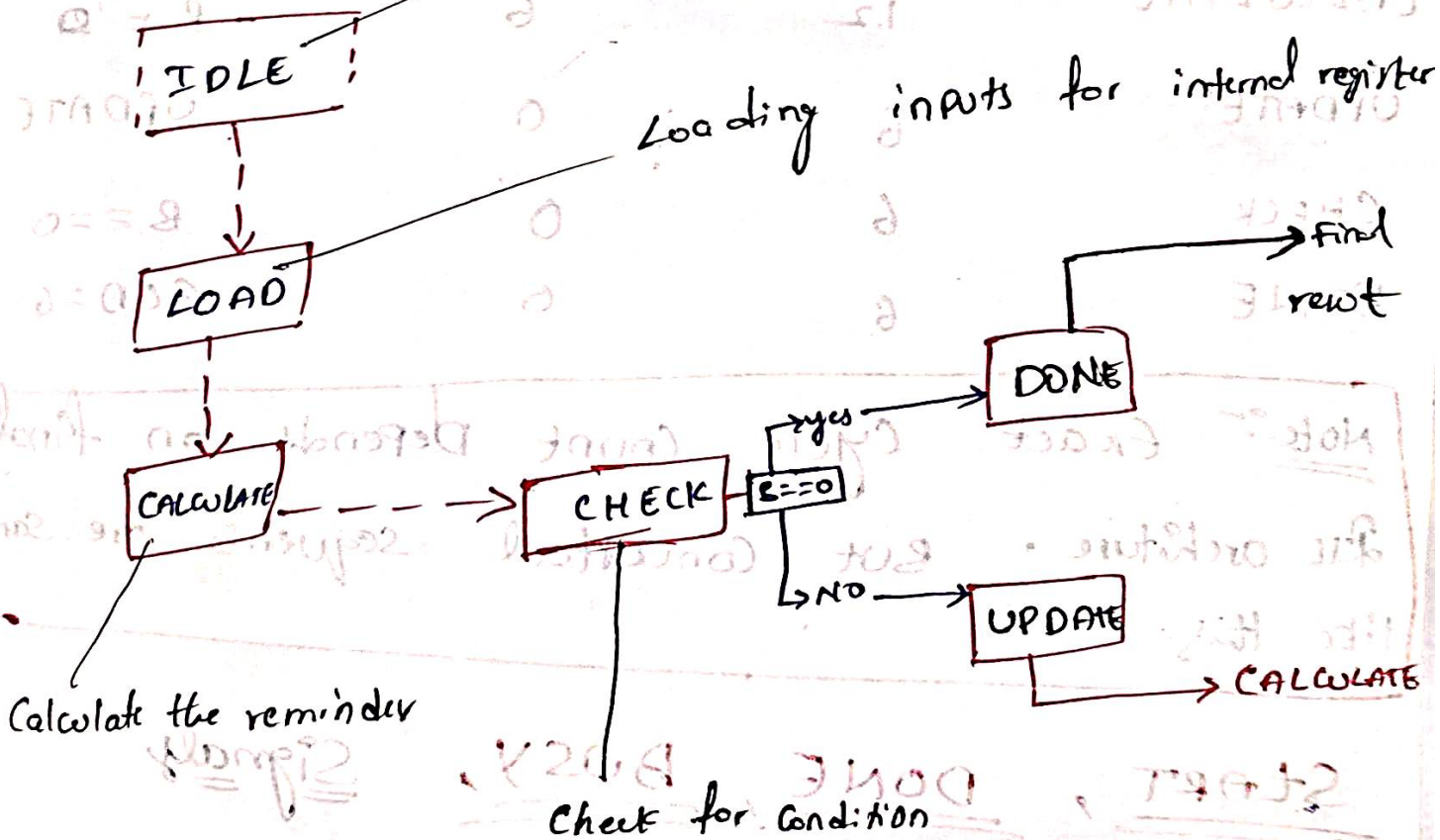
$$R = A \% B$$



Practical based on implementation architecture/synthesis may be it different

why Fsm logic

wait for div calculation.



Clock by clock Example

Let's use

$$A = 48$$

$$B = 18$$

State	A = reg
LOAD	48
CALCULATE	48
UPDATE	18
CALCULATE	18
UPDATE	12
CALCULATE	12
UPDATE	6
CHECK	6
DONE	6

Reg	operation
18	Load i/p
18	R = 12
12	UPDATE
12	R = 6
6	UPDATE
6	R = 0
0	UPDATE
0	B = 0
0	on GCD = 6

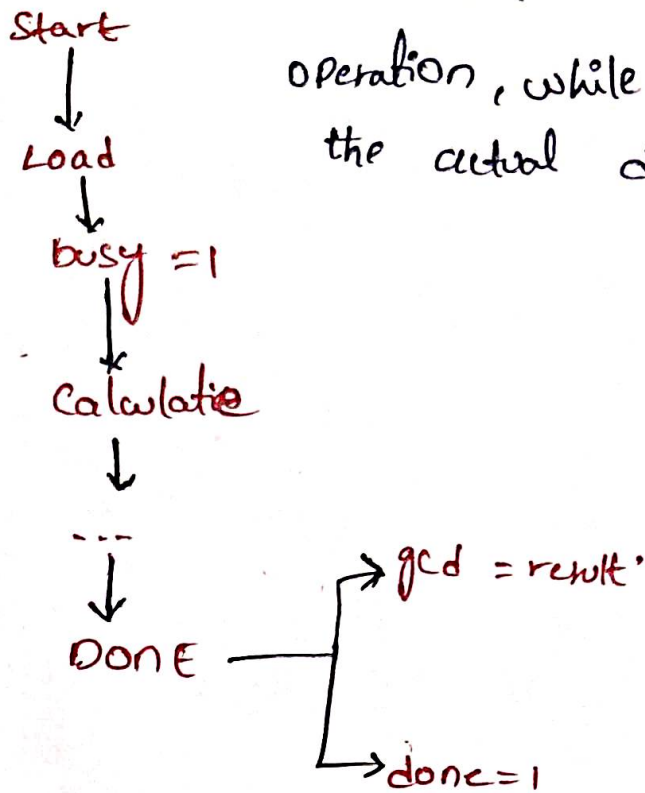
Note:- Exact cycle count depends on final RTL architecture. But conceptual sequence are same like this

START, DONE, BUSY, Signals

Start ———→ Sending request to DUT for startin Calc
 DONE ———→ Calculation Complete.
 busy ———→ Any current DUT calculation is happen.

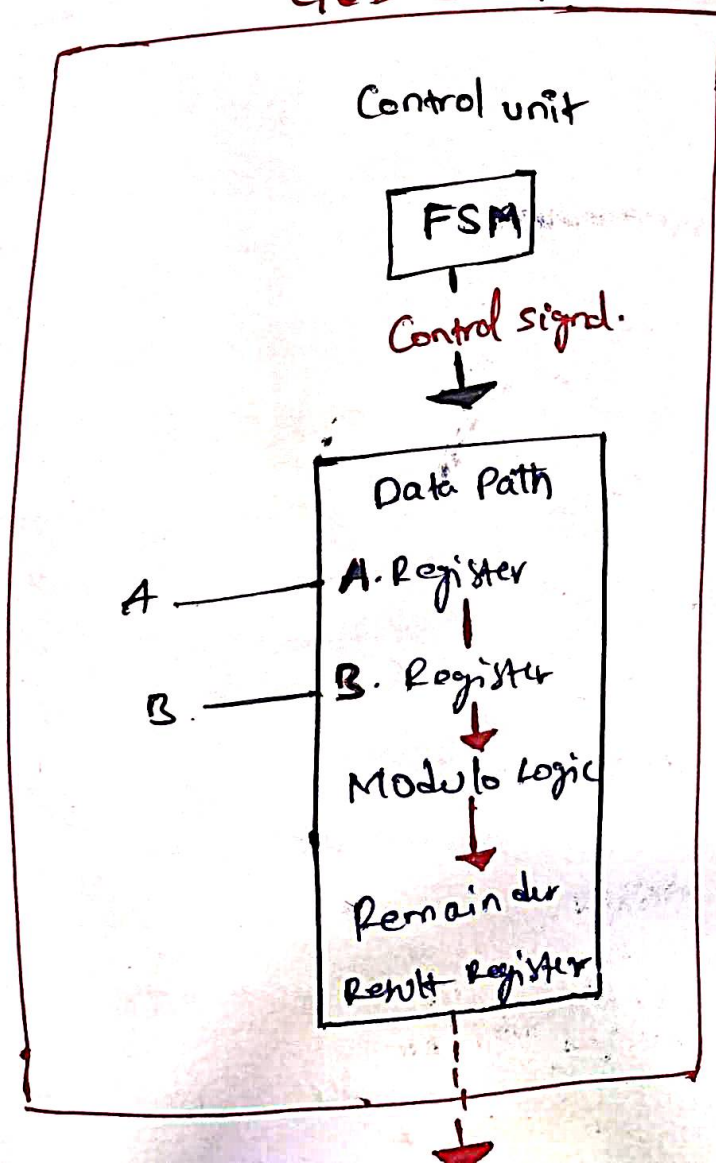
Ex:-

* Fsm Controls the sequence of operation, while the data path performs the actual data processing.



Final Project architecture

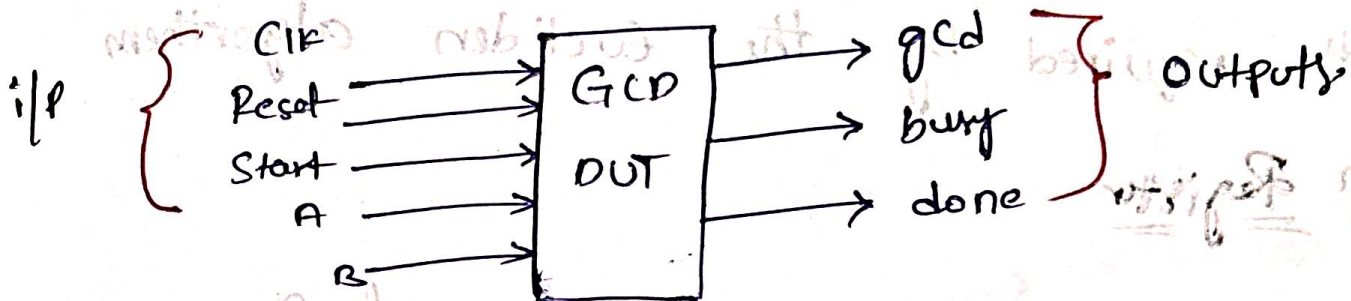
GCD DUT



* Data Path performs the operation, while the Control unit decides when and how the operation is performed.

GCD RTL CODE Theory

GCD RTL Signals



The GCD module uses clock, reset, start, input data, result, and status signals to control and monitor the GCD calculation.

Important Signals

- Clk — Synchronous sequential operation
- Reset — Initialize the OUT to a known state
- Start — Initiates the GCD calculation.
- A, B : — Input operands.
- gcd — Stores the calculated GCD
- busy — Indicates that the calculation is in progress.
- done — Indicates that the calculation has completed

Key Points

GCD Data Path

The GCD datapath stores and processes the operands required by the Euclidean algorithm.

Main Register

- A-reg — Store the current value of A.
- B-reg — Store the current value of B.
- R-reg — Store the remainder $A \% B$.

Main operation

$$R\text{-reg} = A\text{-reg} \% B\text{-reg}$$

$$A\text{-reg} = B\text{-reg}$$

$$B\text{-reg} = R\text{-reg}$$

Termination

if $B\text{-reg} == 0$

$$GCD = A\text{-reg}$$

Important RTL Concept

Non blocking assignment ($<=$) are used for sequential register updates so that all registers update based on their previous values at the clock edge.

Final architune

